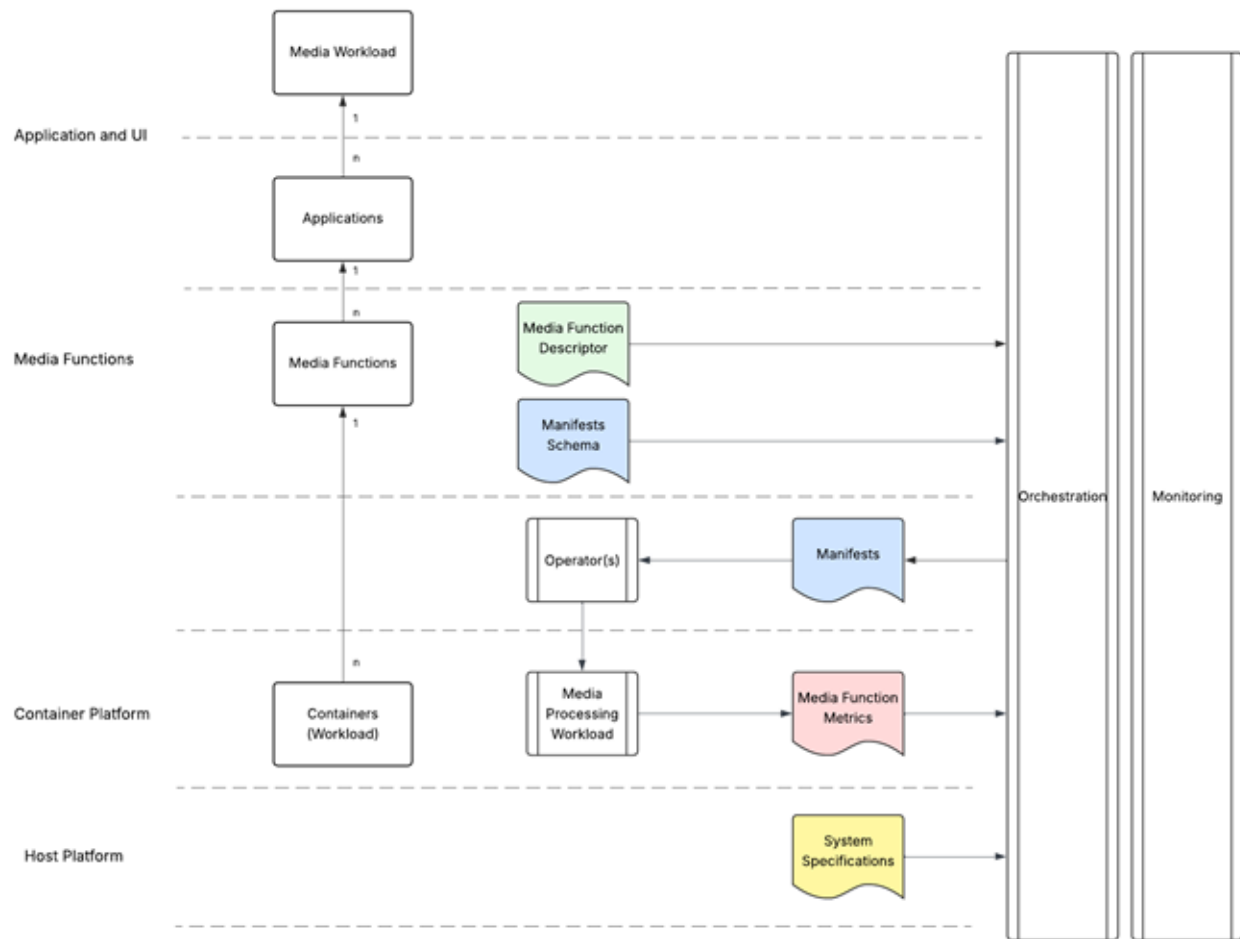




Real-World Deployment Patterns

Media Functions are developed by many vendors, who leverage different deployment patterns. The DMF architecture needs to support a wide range of these, minimizing the additional development cost for vendors to bring their products to DMF platforms.

Basic Media Functions involve:

- Single pod (quite commonly and in the limit, single container) per Media Function instance
- No Media Function-specific shared services

More complex Media Functions can have:

- Either/or
 - Multiple pods per Media Function instance [1..N]
 - Multiple Media Function instances per pod (where a logical Media Function instance is an independently configured session/channel/task, with some upper bounds on the number supported by each deployed pod) [N..1]
- Media Function kind-specific shared context and services (for example, asset storage)
- Media Function kind-specific cluster-wide configuration and services (for example, site license server)

Real-World Deployment Patterns

Media Function developers must be able to package their software components using their preferred virtualization technology:

- Container-based - recommended for resource efficiency and orchestration tooling
- VM-based
 - Linux
 - Windows(!)

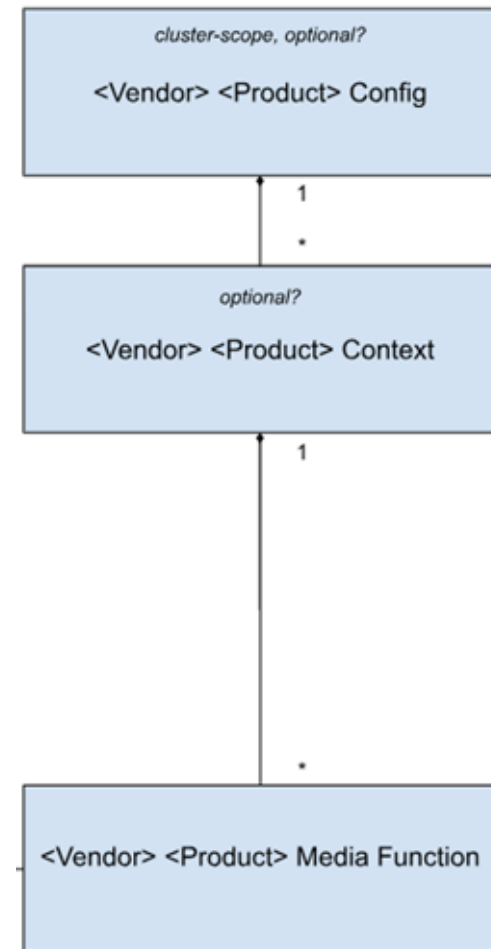
Resource requests/limits/claims must be applied to individual containers (or VMs)

- Logical Media Functions do not provide sufficient granularity

Proposed Domain Model

- Media Function
 - Represents a particular configured instance or session of a well defined unit of functionality that produces and/or consumes a tightly coupled group of one or more live media streams.
 - [JT-DMF FC working group] The particular vendor/product Media Function specification defines the input and output characteristics, such as control type (NMOS...), transport (MXL...), and media format type and properties, that can be used to plan connections between compatible inputs and outputs.
- Media Function Context
 - Represents a management group of Media Functions that could be used to keep multiple live productions separate, for example, with separate asset storage.
- Media Function Config
 - Represents cluster-wide configuration and deployment of high-level components, such as a database or license server, that can or should serve multiple Contexts.

Compute Resource Management could be appropriate at any of these levels.



Reconciliation in Kubernetes

From [Why Implementing Kubernetes Operators Is a Good Idea! | Kubermatic](#)

